If-Else And Else-If Statements

Generated Aug 12, 2026 8:18 AM

What We'll Learn

By the end of this lesson, students can:

- Explain how `if-else` statements choose between two actions.
- Use `else-if` statements to check multiple conditions.
- Arrange conditions in the correct order.
- Trace the output of Java programs with multiple decisions.

Before We Start

Consider how a student's score may be classified:

- A score of `75` or higher is passed.
- A score below `75` is failed.

What if the program needs more than two results, such as `Excellent`, `Passed`, and `Failed`?

Java uses `if-else` and `else-if` statements to choose between different possible actions.

Let's Understand

The `if-else` Statement

An `if-else` statement allows a program to choose between two possible actions.

Syntax:

```
if (condition) {
    // Runs when the condition is true
} else {
    // Runs when the condition is false
}
```

Example:

```
int score = 82;

if (score >= 75) {
    System.out.println("Result: Passed");
} else {
    System.out.println("Result: Failed");
}
```

Output:

```
Result: Passed
```

The condition `score >= 75` is `true`, so the statement inside the `if` block runs.

If the condition were `false`, the statement inside the `else` block would run.

Only one of the two blocks will run.

The `else-if` Statement

An `else-if` statement is used when a program needs to check multiple conditions.

Syntax:

```
if (condition1) {
    // Runs when condition1 is true
} else if (condition2) {
    // Runs when condition2 is true
} else {
    // Runs when all previous conditions are false
}
```

Example:

```
int score = 85;

if (score >= 90) {
    System.out.println("Excellent");
} else if (score >= 75) {
    System.out.println("Passed");
} else {
    System.out.println("Failed");
}
```

Output:

```
Passed
```

Java checks the conditions from top to bottom:

1. It checks whether `score >= 90`.
2. The first condition is `false`, so Java checks `score >= 75`.
3. The second condition is `true`, so Java displays `Passed`.
4. The remaining blocks are skipped.

Checking Multiple Conditions

More than one `else if` block may be used.

```
int score = 88;

if (score >= 90) {
    System.out.println("Excellent");
} else if (score >= 80) {
    System.out.println("Very Good");
} else if (score >= 75) {
    System.out.println("Passed");
} else {
    System.out.println("Failed");
}
```

Output:

```
Very Good
```

Java stops checking as soon as it finds a condition that is `true`.

Arranging Conditions Correctly

When checking number ranges, conditions should usually be arranged from the highest value to the lowest value.

Correct order:

```
if (score >= 90) {
    System.out.println("Excellent");
} else if (score >= 80) {
    System.out.println("Very Good");
} else if (score >= 75) {
    System.out.println("Passed");
} else {
    System.out.println("Failed");
}
```

Incorrect order:

```
if (score >= 75) {
    System.out.println("Passed");
} else if (score >= 80) {
    System.out.println("Very Good");
} else if (score >= 90) {
    System.out.println("Excellent");
}
```

In the incorrect example, a score of `95` satisfies `score >= 75` immediately. The program displays `Passed` and skips the remaining conditions.

How Java Makes the Decision

When Java reaches an `if-else-if` statement:

1. It checks the first condition.
2. If the condition is `true`, it runs that block and skips the rest.
3. If the condition is `false`, it checks the next condition.
4. It continues until it finds a condition that is `true`.
5. If all conditions are `false`, it runs the `else` block.

Let's Practice

Create a Java program named `AgeClassifier`.

The program must:

1. Store a person's age in an integer variable.
2. Display `Child` when the age is below `13`.
3. Display `Teenager` when the age is from `13` to `19`.
4. Display `Adult` when the age is `20` or higher.
5. Use an `if-else-if` statement.
6. Test the program using different age values.